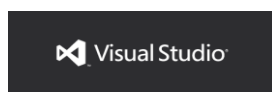


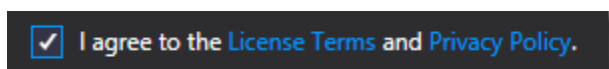
I. ИНСТАЛАЦИЯ И КОНФИГУРИРАНЕ НА РАБОТНАТА СРЕДА. ВЪВЕЖДАНЕ, КОМПИЛАЦИЯ И ИЗПЪЛНЕНИЕ НА C# ПРОГРАМИ

1. Запознава студентите със средата за разработка Visual Studio .NET и езика за програмиране C#, като дава възможност на всеки студент:
2. да разбере как се инсталира средата за разработка Visual Studio .Net,
3. да разбере как се създава, кодира и компилира проект в средата Visual Studio .Net чрез програмния език C#.
4. да разбере как се добавят контроли в проект, как се извършва промяна в свойствата им чрез директен достъп от меню Properties и чрез програмен код.
5. да разбере начина на деклариране на променливи в езика C#, добавяне на коментари, конвертиране на типове, обработка на изключения чрез try ... catch ... finally конструкция.

1. Инсталация



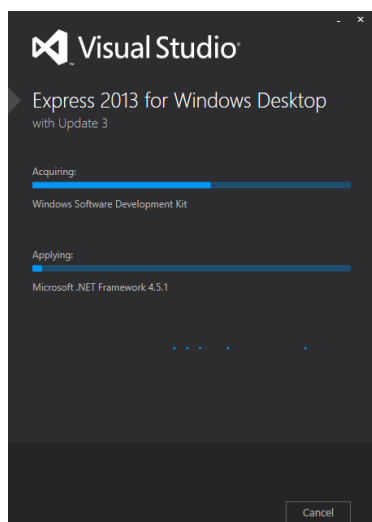
Фиг.1.1. Екран за зареждане в средата Visual Studio при инсталиране



Фиг.1.2. Съгласие за лиценз и политика за поверителност

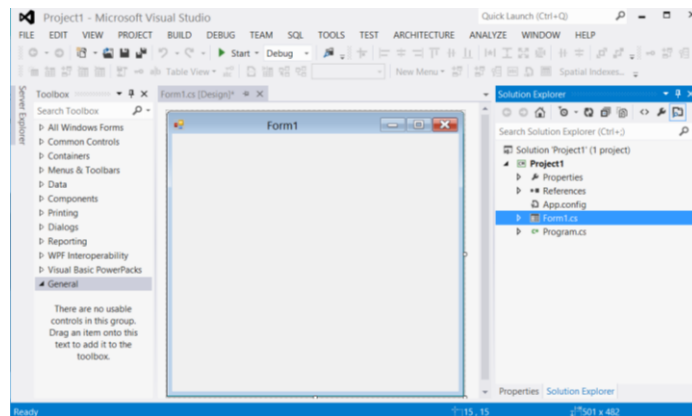


Фиг.1.3. Бутон за инсталиране



Фиг.1.4. Екран визуализиращ процеса по инсталиране на Visual Studio

2. Създаване на приложения



Фиг.1.5. Създаване на нов проект в C#

Свойства (Properties) – при разгъване се вижда файла AssemblyInfo.cs. Той е файл с метаданни, който съдържа информация като: версия, име на продукта, име на компанията, авторски права и други.

Референции (References) - съдържа списък на всички библиотеки, изисквани от проекта. По подразбиране има няколко основни библиотеки от .NET Framework.

App.config - конфигурационен файл използван от .NET Framework.

Program.cs - това е мястото, където е разположен C# кода.

Form1. cs. – този файл се генерира при стартиране на проекта и включва основните пространства от имена от FCL, като: System, System.Text и други описани по-горе, както и конструктор на класа public Form1(), който се изпълнява, при стартиране на класа Form1:

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace Project1
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
    }
}
```

Пример:

```
private void button1_Click(object sender, System.EventArgs e)
{
    System.String cs = @"Data Source=(LocalDB)\v11.0; AttachDbFilename=
C:\Users\My\Documents\Experiment1\ Database1.mdf;Integrated Security=True";
}
```

Това може да стане и чрез добавя на “using” и именното пространство System:

using System;

```
private void button1_Click(object sender, EventArgs e
```

```
{
```

```
String cs = @"Data Source=(LocalDB)\v11.0;AttachDbFilename=
```

```
C:\Users\My\Documents\Experiment1\Database1.mdf;Integrated Security=True";
```

```
}
```

3. Типове данни. Преобразувания на типове в C#

Таблица 1.1. Вградените типове данни в C#

Тип на данни	Значение	Обхват
byte	Байт	От 0 до 255
sbyte	Байт със знак	От -128 до 127
short	Малко цяло число със знак	От -32768 до 32767
ushort	Малко цяло число без знак	От 0 до 65,535
int	Цяло число	От -2147483648 до 2147483647
uint	Цяло число без знак	От 0 до 4294967295
long	Дълго число	От -9223372036854775808 до 9223372036854775807
ulong	Дълго число без знак	От 0 до 18446744073709551615
float	Плаваща десетична запетая	Грубо от -3.4e38 до 3.4e38
double	Плаваща десетична запетая с двойна точност	Грубо от -1.8e308 до 1.8e308
decimal	С по-висока точност и по-малък обхват, отколкото типовете за плаваща запетая	-
char	Символ	Отделен символ Unicode. Символите Unicode използват 16 бита, за да записват данни за текст в азбуки като арабската, кирилицата, гръцката и тайската освен римската (латинска) азбука
string	Текст	Низ от символи Unicode
bool	Булева	Може да бъде true или false
object	Обект	Може да сочи към почти всичко

Пример:

```
int a = 100;
```

```
double b = a;
```

Пример:

```
double a = 3.4;
```

```
double b = 3.3;
```

```
double c = 3.3;
```

```
int result = (int)a+(int)b+(int)c;
```

Резултата е: 9

Ако се приложи неявно конвертиране т.е

```
result = a + b + c
```

Резултата е 10

Таблица 1.2.

Конвертиране в C#	Предназначение
Неявни преобразувания (Implicit conversions)	Неявни преобразувания: не се изисква специален синтаксис, защото преобразуването винаги е успешно и няма да бъдат загубени данни. Примерите включват преобразувания от по-малки в по-големи интегрални типове и преобразувания от производни класове в базови класове. [2]
Явни преобразувания (Explicit conversions (casts))	Явни преобразувания (кастване): Изричните преобразувания изискват израз на гласове. Предаването е необходимо, когато при преобразуването може да бъде загубена информация или когато преобразуването може да не успее по други причини. Типичните примери включват цифрово преобразуване във вид с по-малка точност или по-малък диапазон и преобразуване на екземпляр от базов клас във производен клас. [2]
Конфигурирани от потребителя преобразувания (User-defined conversions)	Конфигурирани от потребителя преобразувания: дефинираните от потребителя преобразувания се извършват чрез специални методи, които могат да бъдат определени, за да се активират явни и неявни преобразувания между персонализирани типове, които нямат базисни класови отношения. [2]
Конверсии с помощни класове (Conversions with helper classes)	Конверсии с помощни класове: за да се извърши преобразуване между несъвместими типове, като цели числа и System.DateTime обекти, или шестнадесетични низове и байтови масиви, може да се използва класа System.BitConverter, класа System.Convert и методите Parse вградени цифрови типове, като Int32.Parse. [2]

Пример:

```
int number = 12345678;
```

```
long bigNumber = number;
```

Пример:

```
double x = 150.45;
int y;
y = (int)x;
System.MessageBox.Show("y="+y);
```

Резултата е: 150

4. Обработка на изключения

Изключенията се управляват в try ... catch ... finally блок.

```
try
{
    // Statements which may throw exceptions
}
catch (Exception ex)
{
    // Exception caught and handled here
}
finally
{
    // Statements always executed after the try/catch blocks
}
```

5. Задачи за изпълнение

Задача 1. Да се създаде проект на C#, който съдържа бутон, при кликване върху, който се извежда съобщение: „Това е вашия първи проект на C#“. Да се променят свойствата на използваните контроли съобразно Таблица 1.3.

За реализиране на поставената цел се създава нов проект с име Project1, добавя се контролата **button1**. Променят се свойствата на контролата, както и тези на **Form1**, съгласно указанията в таблица 1.3., като се използва меню Properties:

Таблица 1.3.

Контрола	Свойство	Стойност	Описание
Form1	Text	Първи проект	Задава текст на формата
Form1	BackColor	По избор	Задава цвят на формата
button1	Text	Поздравления!	Задава текст на бутона
button1	BackColor	По избор	Задава цвят на бутона
button1	ForeColor	По избор	Задава цвят на текста

Към проекта се добавя метод `private void button1_Click` с аргументи `object sender, EventArgs e`, за извеждане на съобщение:

```
private void button1_Click(object sender, EventArgs e)
{
    MessageBox.Show("\t Това е вашия първи \n \n \t \t с проект n \n \t \t \t на C#"); }

```

Задача 2. Към *Project1* от *задача1* се добавят, следните контроли: **label1**, **button2** и **button3**. Променят се свойствата им, както е указано в Таблица 1.4.

Таблица 1.4.

Контрола	Свойство	Стойност
Form1	BackColor	Променя се чрез програмен код
button2	Text	Red
button3	Text	Blue
label1	Text	Променя се чрез програмен код
label1	ForeColor	Променя се чрез програмен код
label1	Font	Променя се чрез програмен код

```
private void button2_Click(object sender, EventArgs e)
{
    this.BackColor = Color.Red;
    //this.BackColor = Color.FromArgb(204, 0, 0);
}

private void button3_Click(object sender, EventArgs e)
{
    this.BackColor = Color.Blue;
    //this.BackColor = Color.FromArgb(0, 102, 204);
}

private void Form1_Load(object sender, EventArgs e)
{
    this.BackColor = Color.Black;
    // this.BackColor = Color.FromArgb(0, 0, 0);
    label1.Text = " Кликнете върху бутонa \n за да видите как работи той.";
    label1.Font = new Font("Arial", 14);
    label1.ForeColor = Color.White;
}
}

```

Задача 3. Като се използва *Project1* от *задача2* да се напише функция, която съобразно цвета на бутонa да променя цвета на фона на формата и заглавия и текст. Нека текста в заглавието да показва кой е обекта (т.е бутон *sender*), предизвикал събитието. Да се добавят бутони за допълнителна промяна на цвета.

За целта се създава функция `ChangeFormColor`, която се извиква на всеки от бутоните и предизвиква промяна в цвета на фона на формата, съобразно цвета на бутона върху, който се кликва.

```
private void ChangeFormColor(object sender, EventArgs e)
```

```
{ if (sender is Button)
    { this.BackColor = (sender as Button).BackColor;
      this.Text = sender.ToString();
    }
}
```

```
private void button2_Click(object sender, EventArgs e)
```

```
{
    ChangeFormColor(sender, e);
}
```

```
private void button3_Click(object sender, EventArgs e)
```

```
{
    ChangeFormColor(sender, e);
}
```

Задача 4. Да се създаде елементарен калкулатор чрез, който се извършва аритметично действие събиране.

За целта се създава нов проект `Project2`. Добавят се контролите: **textBox1**, **textBox2**, **textBox3** и **button1**. Променят се свойствата им, както е указано в Таблица 1.5.

Таблица 1.5.

Контрол	Свойство	Стойност
Form1	Text	Calculator
Form1	Font	Променя се чрез програмен код
textBox1	Text	Променя се чрез програмен код
textBox2	Text	Променя се чрез програмен код
textBox3	Text	Променя се чрез програмен код
textBox3	Enabled	False
button1	Text	Add

Добавя се метод `button1_Click`, който изпълнява аритметично действие събиране. В тялото на метода се декларират 2 променливи `a` и `b` от тип `double`. На първата променлива се присвоява стойността от първото текстово поле, а на втората от второто. Тъй като тези полета са текстови (т.е. от тип `string`), следва да се конвертират към `double`.

```
double a, b;
```

```
a = Double.Parse(textBox1.Text);
```

```
b = Double.Parse(textBox2.Text);
```

Резултатът от пресмятането на двете променливи е от тип `double`. Той се записва в `textBox3`, като се преобразува (конвертира) от `double` към `string`, като се използва `ToString()`:

```
textBox3.Text = (a + b).ToString();
```

За прецизиране на резултата, се изследват входните данни, ако са некоректно въведени се извежда съобщение за грешка (по подразбиране тази грешка е системна). За предотвратяване (улавяне) на грешката се прилага конструкцията `try...catch`, като в частта `catch` се добавя подходящо съобщение:

```
MessageBox.Show("Въведете коректни данни!");
```

Като допълнителна функционалност към кода може да се добави изтриване на входните и изходните данни от текстовите полета. Например:

```
textBox1.Clear();
```

```
textBox2.Clear();
```

```
textBox3.Clear();
```

Окончателният вариант на метода `button1_Click` (с допълнителните функционалности) придобива вида:

```
private void button1_Click(object sender, EventArgs e)
{
    double a, b;
    try
    {
        a = Double.Parse(textBox1.Text);
        b = Double.Parse(textBox2.Text);
        textBox3.Text = (a + b).ToString();
    }
    catch
    {
        MessageBox.Show("Въведете коректни данни!");
        textBox1.Clear();
        textBox2.Clear();
        textBox3.Clear();
    }
}
```

Задача 5. *Като се използва Project2 от задача4 да се реализират по аналогичен начин, аритметичните действия (изваждане, умножение и деление).*

7. Задачи за самостоятелна работа

Задача 1. *Да се напише функция `AddNumbers()`, която връща сумата от пресмятане на две числа в текущия клас `Form1` и записва резултата в трета променлива, като резултата от пресмятането се визуализира в `textBox3`.*

```
public double AddNumbers(double a, double b)
```

```
{
```

```
return a + b;
}
```

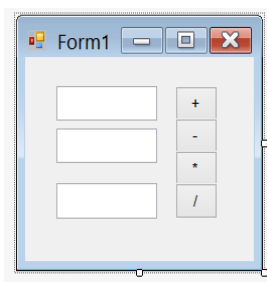
Задача 2. Да се създаде нов клас “Calculator.cs.”, в който да се добави функцията от задача1 от задачите за самостоятелна работа. Да се извика и изпълни в метод `button1_Click` в класа `Form1`. Резултата да се изведе в `textBox3`.

Да се използва синтаксиса за създаване на нов обект:

```
ClassName WhatNameYouWantToUse = new ClassName();
```

Задача 3. Допълнете задача1 от задачите за самостоятелна работа, като добавите нови бутони за функциите за пресмятане на разлика, частно и произведение на две числа.

Задача 4. Да се оптимизира кода в задача3 от задачите за самостоятелна работа, като се разработи функция `Calculate()`, която да извършва проверка за текста на всеки бутон (нека всеки бутон отговаря на конкретно аритметично действие +,*,-,/). В зависимост от надписа върху бутона да се извиква съответната функция за пресмятане.



Фиг.1.6. Интерфейс на елементарен калкулатор 1

За реализиране на поставената задача е необходимо да се създаде клас `class Calc`, в него се добавят 4 функции за всяко аритметично действие. В `class Form1` се създава метод `CharacterSearch(object sender, EventArgs e)` за извършване на пресмятането. Този метод присвоява прочетените от текстовите полета 2 и 3 стойности на две променливи. След това в зависимост от текста върху всеки от бутоните във `Form1`, се извиква съответната функция за извършване на аритметичното действие. Резултата от пресмятането се записва в текстово поле 3 предназначено за изходни данни. Този метод се извиква във всеки един от бутоните, чрез които ще става пресмятането.

Примерно решение:

```
namespace UP1_4S
```

```
{
    class Calc
    {
        public static double Addition(double a, double b)
        {
            return a + b;
        }
        public static double Subtraction(double a, double b)
        {
            return a - b;
        }
        public static double Multiplication(double a, double b)
```

```

    {
        return a * b;
    }
    public static double Division(double a, double b)
    {
        return a / b;
    }
}

// Метода void CharacterSearch(object sender, EventArgs e) се добавя във Form1
private void CharacterSearch(object sender, EventArgs e)
{
    double a, b;
    try
    {
        a = Double.Parse(textBox1.Text);
        b = Double.Parse(textBox2.Text);
        if (sender is Button)
        {
            if ((sender as Button).Text=="+")
                textBox3.Text = Calc.Addition(a, b).ToString();
            if ((sender as Button).Text=="-")
                textBox3.Text = Calc.Subtraction(a, b).ToString();
            if ((sender as Button).Text=="*")
                textBox3.Text = Calc.Multiplication(a, b).ToString();
            if ((sender as Button).Text=="/")
                if (b != 0)
                    textBox3.Text = Calc.Division(a, b).ToString();
            else
            {
                MessageBox.Show("Не се дели на 0!");
                Clear();
            }
        }
    }
    catch
    {

```

```

        MessageBox.Show("Въведете коректни данни!");
        Clear();
    }
}

private void button1_Click(object sender, EventArgs e)
{
    CharacterSearch(sender, e);
}

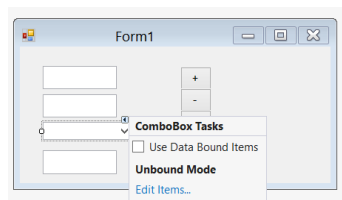
private void button2_Click(object sender, EventArgs e)
{
    CharacterSearch(sender, e);
}

private void button3_Click(object sender, EventArgs e)
{
    CharacterSearch(sender, e);
}

private void button4_Click(object sender, EventArgs e)
{
    CharacterSearch(sender, e);
}
}
}

```

Задача 5. Да се модифицира задача3, като се добави *comboBox*, към който се добавят операциите за събиране, изваждане, умножение и деление, например: "+", "-", "*" и "/". При стартиране на проекта, да се показва в *comboBox*-а първият елемент от него, да се избере подходящо свойство на *comboBox*-а, така, че да не може да се променят стойностите в него. Да се ограничат допустимите символи при въвеждане, така, че да включват числа от 0 до 9, запетая, минус и *Backspace*.



Фиг.1.7. Интерфейс на елементарен калкулатор 2

Забележка: Добавяне на стойности в контролата *comboBox* става по два начина ръчно след кликване с десен бутон върху контролата се появява прозорец един от надписите е *Edit Items*, при кликване върху него се отваря лист, в който богат да бъдат добавени стойности, след всяка нововъведена стойност се поставя *Enter*. Втори начин за добавяне е посредством код:

```
comboBox1.Items.Add("стойност");
```

по аналогичен начин се добавя всяка следваща стойност.

За да могат стойностите въведени в comboBox да се появяват със стартиране на програмата, кода с тяхното въвеждане в comboBox се поставя във метода Form_Load. За да се покаже конкретна стойност от списъка от стойности в comboBox е необходимо да се добави код, който да укаже кой е селектирания индекс, например, ако трябва да се покаже първият се записва: comboBox1.SelectedIndex = 0;

Подобно на задача 4 от задачите за самостоятелна работа се добавя метод void CharacterSearch(object sender, EventArgs e), чрез който се извършва пресмятането. И този метод присвоява прочетените от текстовите полета 2 и 3 стойности на две променливи. Но след това вместо да прочете текста върху всеки от бутоните във Form1, прочита текста от списъка на comboBox. След това извика съответната функция за извършване на аритметичното действие, а резултата от пресмятането се записва в текстово поле 3 изходния резултат. Втората промяна е, че този метод, не се извиква във всеки един от бутоните (т.е. в метода Click), а се извиква в методите void textBox_TextChanged(object sender, EventArgs e) за всеки от бутоните, като се поставя предварително условие да има въведени стойности в текстовите полета, иначе ще се генерира грешка например:

```
private void textBox1_TextChanged(object sender, EventArgs e)
{
    if (textBox1.Text != "" && textBox2.Text != "")
        CharacterSearch(sender, e);
}
```

В задачата има условие за допустимите символи при въвеждане, така, че да включват числа от 0 до 9, запетая, минус и Backspace. Един от начините за постигане на това ограничение е посредством метода textBox_KeyPress(object sender, KeyPressEventArgs e). Като посредство e.KeyChar се указва кой символ ще бъде допуснат:

```
private void textBox_KeyPress(object sender, KeyPressEventArgs e)
{
    {
        if (!char.IsDigit(e.KeyChar) && e.KeyChar != (char)Keys.Back && e.KeyChar != '.' &&
e.KeyChar != '-')
        {
            e.Handled = true; textBox1.Focus(); // фокуса се прехвърля към текстово поле 1 и
курсора ще мига в това поле
        }
    }
}
```

По този начина програмата изключва всички останали извън посочените символи.

Примерно частично решение:

namespace UP1_5S

```
{
private void Form1_Load(object sender, EventArgs e)
{
    comboBox1.Items.Add("+");
    comboBox1.Items.Add("-");
    comboBox1.Items.Add("*");
    comboBox1.Items.Add("/");
}
```

```

        comboBox1.SelectedIndex = 0;
    }
private void CharacterSearch(object sender, EventArgs e)
{
    double a, b;
    try
    {
        a = Double.Parse(textBox1.Text);
        b = Double.Parse(textBox2.Text);
        if (comboBox1.Text == "+")
            textBox3.Text = Calc.Addition(a, b).ToString();
        if (comboBox1.Text == "-")
            textBox3.Text = Calc.Subtraction(a, b).ToString();
        if (comboBox1.Text == "*")
            textBox3.Text = Calc.Multiplication(a, b).ToString();
        if (comboBox1.Text == "/")
            if (b != 0)
                textBox3.Text = Calc.Division(a, b).ToString();
            else
            {
                MessageBox.Show("Не се дели на 0!");
                Clear();
            }
    }
    catch { MessageBox.Show("Въведете стойност!"); }
}
private void textBox1_TextChanged(object sender, EventArgs e)
{
    if (textBox1.Text != "" && textBox2.Text != "")
        CharacterSearch(sender, e);
}
private void textBox2_TextChanged(object sender, EventArgs e)
{
    if (textBox1.Text != "" && textBox2.Text != "")
        CharacterSearch(sender, e);
}
private void comboBox1_SelectedIndexChanged(object sender, EventArgs e)

```

```

        {
            if (textBox1.Text != "" && textBox2.Text != "")
                CharacterSearch(sender, e);
        }
private void textBox1_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsDigit(e.KeyChar) && e.KeyChar != (char)Keys.Back && e.KeyChar != '.' &&
e.KeyChar != '-')
    {
        e.Handled = true; textBox1.Focus();
    }
}
private void textBox2_KeyPress(object sender, KeyPressEventArgs e)
{
    {
        if (!char.IsDigit(e.KeyChar) && e.KeyChar != (char)Keys.Back && e.KeyChar != '.' &&
e.KeyChar != '-')
        {
            e.Handled = true; textBox1.Focus();
        }
    }
}

```

Задача 6: Да се създаде приложение от тип *Windows Forms*. Да се добави един бутон с надпис „even or odd”, един етикет и едно текстово поле. Да се провери въведеното в текстовото поле число дали е четно или нечетно. Да се направи проверка за некоректни данни, като се изведе съобщение „Input string is not in correct format”, като се използва конструкцията *try...catch*. При стартиране на приложението да се изведе съобщение, което информира потребителя за целта на приложението. Да се добави бутон за изход от приложението, с надпис *Exit* или *Close*. Приложението да се затваря, само ако потребителя потвърди това.

Забележка: За извеждане на съобщения в C# се използва клас *MessageBox*. Този клас има метод *Show*, който може да приема различни параметри, най-използваните от които са:

```

public static DialogResult Show(
string text, string caption,
MessageBoxButtons buttons, MessageBoxIcon icon,
MessageBoxDefaultButton defaultButton, MessageBoxOptions options)

```

Описание на параметрите:

text: текст на съобщението

caption: текст на заглавието на съобщението

buttons: бутони, използвани в полето за съобщения, приема една от стойностите изброени в `MessageBoxButtons`: **AbortRetryIgnore**: три бутона за избор **Abort** (Отмяна), **Retry** (Повтори), **Ignore** (Пропусни)

OK: един бутон за избор **OK**

OKCancel: два бутона **OK** и **Cancel** (Отмяна)

RetryCancel: два бутона **Retry** (Повтори) и **Cancel** (Отмяна)

YesNo: два бутона **Yes** и **No**

YesNoCancel: три бутона **Yes**, **No** и **Cancel** (Отмяна)

Методът `MessageBox.Show` връща обект `DialogResult`, с който показва кой бутон в полето за съобщение е натиснал потребителят.

```
DialogResult result = MessageBox.Show("Четно!", "Четно или нечетно число?",  
MessageBoxButtons.YesNo);
```

```
{  
    if (result == DialogResult.Yes)  
        this.Close();  
    else Application.Exit();  
}
```